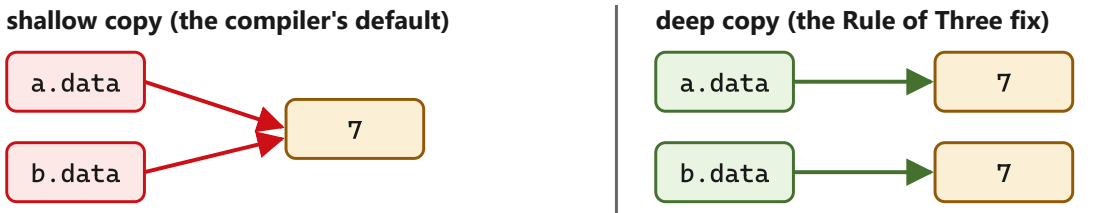
Copy Control and Operators

What copying an object really does: shallow and deep copies, the Rule of 3, 5 and 0, `explicit` and conversion operators, copy-and-swap, and operator overloading. Assumes Classes and Objects.

Copying and the Rule of 3, 5 and 0

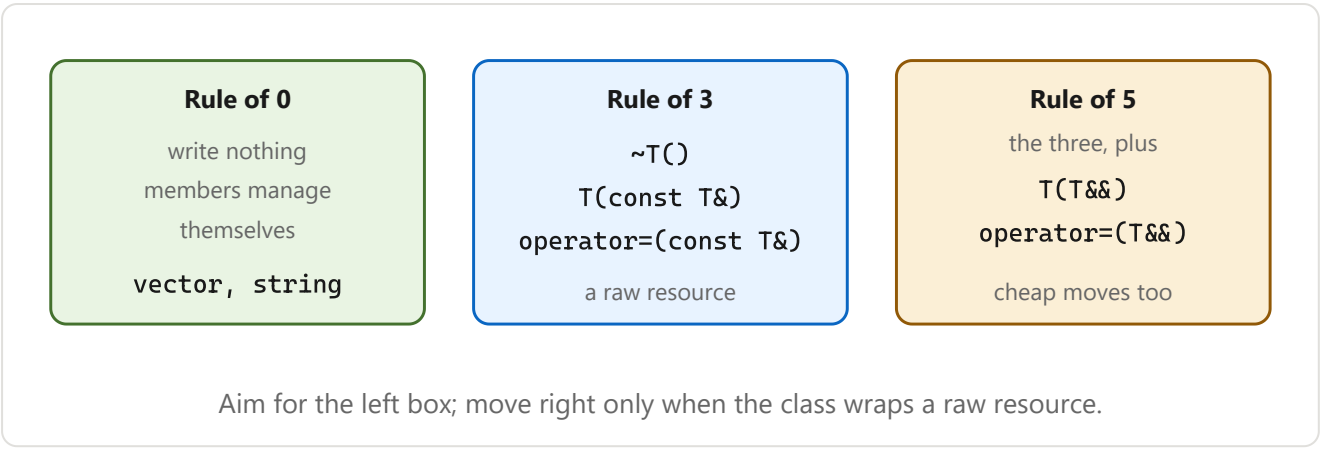
The compiler generates copy operations that copy every member. For a raw pointer that copies only the *address*: a **shallow copy**, and a bug:



Copying the pointer shares the block; copying what it points to separates them.

Fix: a **deep copy**. **Rule of Three**: a custom destructor, copy constructor or copy assignment means you need all three.

```
class Buffer {
    int* data_;
public:
    Buffer(int v) : data_(new int(v)) {}
    // 1. destructor
    ~Buffer() { delete data_; }
    // 2. copy constructor (deep)
    Buffer(const Buffer& o) : data_(new int(*o.data_)) {}
    // 3. copy assignment (deep)
    Buffer& operator=(const Buffer& o) {
        // guard against a = a
        if (this ≠ &o) { *data_ = *o.data_; }
        return *this;
    }
};
```



Aim for the left box; move right only when the class wraps a raw resource.

RULE	WHAT YOU WRITE	WHEN
Rule of 0	none of them	The goal. Hold resources in members that manage themselves: `std::vector`, `std::string`, `std::unique_ptr`.
Rule of 3	destructor, copy ctor, copy assign	The class owns a raw resource and must define copying.
Rule of 5	the 3 plus move ctor and move assign	Same, and you also want cheap moves (see Move Semantics).

```
class NoCopy {
public:
    NoCopy() = default;
    NoCopy(const NoCopy&) = delete;              // forbid copying outright
    NoCopy& operator=(const NoCopy&) = delete;
};
```

What the compiler still generates once you declare something

YOU DECLARE	COPYING	MOVING
nothing	generated	generated
a destructor, a copy constructor or a copy assignment	still generated (but deprecated)	**not generated**: an rvalue silently falls back to a copy
a move constructor or a move assignment	**disabled**	only what you declared

- A class can quietly lose its moves (a later logging destructor turns every move into a copy). Ask for defaults with `= default`; a polymorphic base needs them defaulted too.
- **Avoid `= delete` on a move**: a deleted move still takes part in overload resolution, so an rvalue no longer falls back to the copy and the call fails to compile. For a move-only type, delete the *copy* operations instead.

> IN SHORT "Rule of Zero: `vector` and smart pointers make the generated members correct. Rule of Five only when wrapping a raw resource."

`explicit`, conversion operators and copy-and-swap

```
// implicit: a double silently becomes Meters
struct Meters { double v; Meters(double x) : v(x) {} };
// explicit: must be written out
struct Feet   { double v; explicit Feet(double x) : v(x) {} };
// compiles: surprising
takeMeters(3.0);
// takeFeet(3.0); // ERROR
takeFeet(Feet(3.0));                                    // OK

struct Handle {
    int fd;
    // a conversion operator, but explicit
    explicit operator bool() const { return fd ≥ 0; }
};
// OK: an if is a "contextual" conversion to bool
if (Handle h{3}) { }
// int n = h + 1; // ERROR: no implicit conversion to bool or int
```

- **Mark single-argument constructors `explicit`** unless an implicit conversion is clearly wanted (compiled: `double` converts to `Meters` but not to `Feet`).
- **`explicit operator bool`** is the safe way to say "valid": it works in `if`, `while`, `&&` and `!`, but not in arithmetic or in comparisons with other types (the old "safe bool idiom" is obsolete). The standard streams and `std::optional` use it.
- A non-explicit `operator int()` lets your class take part in arithmetic and overload resolution in ways you did not plan.

Copy-and-swap

```
class Buffer {
    std::size_t n_; int* p_;
public:
    // deep copy (written once)
    Buffer(const Buffer& o);
    friend void swap(Buffer& a, Buffer& b) noexcept { std::swap(a.n_, b.n_);
std::swap(a.p_, b.p_); }
    // by VALUE: the copy is made before we touch *this
    Buffer& operator=(Buffer other) noexcept {
        // steal the copy's resources
        swap(*this, other);
        // `other` dies and frees our OLD resources
        return *this;
    }
};
```

- **Strong exception guarantee:** if the copy throws, it happens before `*this` is touched.
- **Self-assignment safe** without an `if (this ≠ &o)` check (compiled: `b1 = b1` worked).
- One operator serves copy *and* move assignment (Move Semantics). Cost: a move-assignment written by hand can be a little faster.

Operator overloading

Overload operators only when the meaning is obvious (`+` on vectors, `==` on values, `<<` to print).

a + b	=	`operator+(a, b)`
os << v	=	`operator<<(os, v)` — the left operand is the stream, not your class

So a member function (left operand = this) cannot work for <<. Make it a non-member, a friend if it needs private data.

An operator is a function call; the left operand decides whether it can be a member.

```cpp
struct Vec2 {
    double x{}, y{};

    // member: modifies *this
    Vec2& operator+=(const Vec2& o) { x += o.x; y += o.y; return *this; }
    bool operator==(const Vec2& o) const { return x == o.x && y == o.y; }
};
// non-member built from +=
Vec2 operator+(Vec2 a, const Vec2& b) { return a += b; }

// MUST be non-member:
std::ostream& operator<<(std::ostream& os, const Vec2& v) {
    // the left operand is the stream
    return os << '(' << v.x << ", " << v.y << ')';
}
```

- Non-mutating binary operators (`+`, `==`): **non-member**, so both sides convert. `+=`: member.
- Keep usual semantics: `==` with `≠`, `<` a strict weak ordering. C++20 `⟺` generates all comparisons.
- No new operators; `::` `.` `.*` `?:` cannot be overloaded.

Callable objects

```cpp
// a functor: an object you can call like a function
struct Greater {
    bool operator()(int a, int b) const { return a > b; }
};
// a lambda is a compiler-made functor
std::sort(v.begin(), v.end(), Greater{});
```

`operator()` makes a class callable (the idea behind lambdas, `std::function` and comparators). Conversion operators, and why to mark them `explicit`, are covered earlier, in *explicit, conversion operators and copy-and-swap*.

User-defined literals

```
// kilometres to metres
constexpr long long operator""_km(unsigned long long v) { return
static_cast<long long>(v) * 1000; }
// percent to a fraction
constexpr double operator""_pct(long double v) { return static_cast<double>
(v) / 100.0; }

auto dist = 4_km;         // 4000 (compiled)
auto rate = 25.0_pct;     // 0.25
using namespace std::literals;
// std::string; also 5ms, 2s (chrono) and "view"sv
auto name = "abc"s;
```

A literal suffix is an **operator** you define: `operator""_name`. User suffixes must start with an underscore (the others are reserved for the standard library). Use them for units (kilometres, percent, bytes) so the meaning is in the code, not in a comment.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Copying and the Rule of 3, 5 and 0	C++ Primer	13.1 Copy, assign and destroy	525 to 538
	C++ Primer	13.2 Copy control and resource management	539 to 544
	C++ Primer	13.5 Classes that manage dynamic memory	553 to 559
	C++ Primer	15.7 Constructors and copy control	651 to 658
	Programming: Principles and Practice	17.4 to 17.5 Copying and moving; essential operations	501 to 512
	Effective Modern C++	Item 17, Understand special member function generation	127 to 133
	C++ Move Semantics	3.1 and 3.1.1 to 3.1.2 Move semantics in ordinary classes	53 to 58
	C++ Move Semantics	3.2 Implementing the special copy and move member functions	58 to 66
	C++ Move Semantics	3.3 to 3.4 Rules for special member functions; the Rule of Five or Three	66 to 76
explicit, conversion operators and copy-and-swap	C++ Primer	7.5 Constructors revisited	317 to 328
	C++ Primer	13.3 Swap	545 to 547
	C++ Primer	14.9 Overloading, conversions and operators	608 to 619
Operator overloading	C++ Primer	14.1 to 14.7 Basic concepts through member access	581 to 599
	Programming: Principles and Practice	8.4 to 8.6 Evolving a class (Date), enumerations, operator overloading	238 to 249